

FACE RECOGNITION AND DEEP FAKE DETECTION

IMPLEMENTATION DETAILS

Face Recognition and Deepfake Detection System

1. INTRODUCTION

This document provides detailed instructions for installing, configuring, and executing the **Face Recognition and Deepfake Detection System**. The system is designed to identify individuals using facial recognition techniques and detect deepfake content using deep learning models.

2. SYSTEM OVERVIEW

The system consists of two major modules:

- **Face Recognition Module** – Identifies and verifies individuals using facial features.
- **Deepfake Detection Module** – Detects manipulated or AI-generated fake images/videos.

The application is developed using Python and deployed with a user-friendly interface using Streamlit.

3. SOFTWARE REQUIREMENTS

The following software and tools are required:

- **Operating System:** Windows 10 / Windows 11 / Linux / macOS
- **Programming Language:** Python 3.8 or above
- **IDE/Editor:** VS Code / PyCharm / Jupyter Notebook
- **Libraries & Frameworks:**
 - OpenCV
 - NumPy
 - TensorFlow / Keras
 - scikit-learn
 - face_recognition
 - matplotlib

- Streamlit
-

4. HARDWARE REQUIREMENTS

Minimum Requirements:

- Processor: Intel i5 or equivalent
- RAM: 8 GB
- Storage: 10 GB free space

Recommended:

- Processor: Intel i7 or higher
 - RAM: 16 GB
 - GPU (optional but preferred for faster processing)
-

5. PROJECT SETUP

Step 1: Install Python

Download and install Python from the official website. Ensure Python is added to system PATH.

Step 2: Create Virtual Environment

Open terminal/command prompt and run:

```
python -m venv venv
```

Activate the environment:

- Windows:

```
venv\Scripts\activate
```

- Linux/Mac:

```
source venv/bin/activate
```

Step 3: Install Required Libraries

Install dependencies using:

```
pip install -r requirements.txt
```

If requirements file is not available, install manually:

```
pip install opencv-python numpy tensorflow keras scikit-learn face_recognition matplotlib  
streamlit
```

6. PROJECT STRUCTURE

Project/

|

├─ app.py

├─ predict.py

├─ models/

├─ dataset/

├─ images/

├─ requirements.txt

└─ README.md

7. EXECUTION PROCESS

Step 1: Start the Application

Run the following command:

```
streamlit run app.py
```

Step 2: Open Web Interface

- The application will automatically open in the browser
- If not, go to:

<http://localhost:8501>

Step 3: Using the System

Face Recognition:

1. Upload an image or capture using webcam
2. System detects faces in the image
3. Matches detected faces with stored database

4. Displays identified person name
-

Deepfake Detection:

1. Upload image or video file
 2. System processes input using trained model
 3. Displays result:
 - Real
 - Fake
-

8. DATASET HANDLING

- Face images are stored in the dataset folder
 - Each folder represents a person
 - Deepfake model uses pre-trained datasets
-

9. MODEL DETAILS

- Face Recognition: Uses feature encoding and comparison
 - Deepfake Detection: Uses CNN-based architecture (e.g., EfficientNet or similar)
-

10. OUTPUT RESULTS

The system provides:

- Face detection with bounding boxes
 - Name recognition for known faces
 - Deepfake classification results
 - Confidence scores (if implemented)
-

11. TROUBLESHOOTING

Common issues and solutions:

- **Module not found error:** Install missing libraries using pip
- **Camera not working:** Check device permissions

- **Slow performance:** Use GPU or reduce image size
 - **Model not loading:** Verify file paths
-

12. FUTURE ENHANCEMENTS

- Real-time surveillance integration
 - Cloud deployment using AWS
 - Mobile application integration
 - Improved accuracy using larger datasets
-

13. CONCLUSION

The implementation successfully integrates face recognition and deepfake detection into a single system. The project demonstrates the practical application of machine learning and computer vision in solving real-world problems related to identity verification and digital media authenticity.
